LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 4

ViewModel and Debugging

Oleh:

Muhammad Guntur Ricky Adhitya NIM. 2410817310003

PROGRAM STUDI TEKNOLOGI INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS LAMBUNG MANGKURAT

MEI 2026

LEMBAR PENGESAHAN

LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE

MODUL 4

Laporan Praktikum Pemrograman Mobile Modul 4: ViewModel and Debugging ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan	: Muhammad Guntur Ricky Adhitya
NIM	: 2410817310003

Menyetujui, Asisten Praktikum	Menyetujui, Dosen Penanggung Jawab Praktikum
Muhammad Erza Raihan NIM. 2310817210027	Irham Maulani Abdul Gani, S.Kom., M.Kom. NIP. 199710312025061009

DAFTAR ISI

DAFTAR GAMBAR

DAFTAR TABEL

SOAL 1

Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada

Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut :

a. Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity.

b. Gunakan ViewModelFactory untuk membuat parameter dengan tipe data String di dalam ViewModel

c. Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment

d. Install dan gunakan library Timber untuk logging event berikut:

    a. Log saat data item masuk ke dalam list

    b. Log saat tombol Detail dan tombol Explicit Intent ditekan

    c. Log data dari list yang dipilih ketika berpindah ke halaman Detail

    d. Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi.

Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out.

A. Source Code

Generic Source Code / Module

Tabel 1. CodeforcesProblem Modul 4

```
package com.mobil.modul4{xml/compose}.data


import androidx.annotation.DrawableRes
import androidx.annotation.RawRes
import androidx.annotation.StringRes


data class CodeforcesProblem(
    val problemId: String,
    @StringRes val title: Int,
    @StringRes val description: Int,
    val url: String,
    @RawRes val solutionCode: Int,
    @DrawableRes val img: Int,
)
```

Tabel 2. ProblemRepository Modul 4

```
package com.mobil.modul4{xml/compose}.data


import com.mobil.modul4{xml/compose}.R


object ProblemRepository {
    private val problemList = listOf(
        CodeforcesProblem(
```

```
            problemId = "2220A",
            title = R.string.problem_2220a_title,
            description = R.string.problem_2220a_desc,
                                                    url     =
"https://codeforces.com/contest/2220/problem/A",
                    solutionCode  =  R.raw.problem_2220a_code,  //
Updated
            img = R.drawable.a_blocked
        ),
        CodeforcesProblem(
            problemId = "2209B",
            title = R.string.problem_2209b_title,
            description = R.string.problem_2209b_desc,
                                                    url     =
"https://codeforces.com/contest/2209/problem/B",
                    solutionCode  =  R.raw.problem_2209b_code,  //
Updated
            img = R.drawable.b_array_operation
        ),
        CodeforcesProblem(
            problemId = "2209A",
            title = R.string.problem_2209a_title,
            description = R.string.problem_2209a_desc,
                                                    url     =
"https://codeforces.com/contest/2209/problem/A",
                    solutionCode  =  R.raw.problem_2209a_code,  //
Updated
            img = R.drawable.a_initial_config
        ),
```

```
        CodeforcesProblem(
            problemId = "2125B",
            title = R.string.problem_2125b_title,
            description = R.string.problem_2125b_desc,
                                                    url     =
"https://codeforces.com/contest/2125/problem/B",
                  solutionCode  =  R.raw.problem_2125b_code,  //
Updated
            img = R.drawable.b_left_and_down
        ),
        CodeforcesProblem(
            problemId = "2209C",
            title = R.string.problem_2209c_title,
            description = R.string.problem_2209c_desc,
                                                    url     =
"https://codeforces.com/contest/2209/problem/C",
                  solutionCode  =  R.raw.problem_2209c_code,  //
Updated
            img = R.drawable.c_find_the_zero
        )
    )

    fun getProblemById(id: String): CodeforcesProblem? {
        return problemList.find { it.problemId == id }
    }

          fun   getAllProblems():   List<CodeforcesProblem>   =
problemList
}
```

Tabel 3. DetailState Modul 4

```
package
com.mobil.modul4{xml/compose}.ui.{fragments/screens}.detail


data class DetailState(
    val titleRes: Int = 0,
    val descRes: Int = 0,
    val imgRes: Int = 0,
    val code: String = "",
    val isNotFound: Boolean = false
)
```

Tabel 4. DetailViewModel Modul 4

```
package com.mobil.modul4compose.ui.screens.detail


import android.content.Context
import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.mobil.modul4compose.data.ProblemRepository
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.update
import kotlinx.coroutines.launch
import timber.log.Timber
```

```
class DetailViewModel(
    private val problemId: String,
    private val moduleName: String
) : ViewModel() {

    private val _state = MutableStateFlow(DetailState())
    val state: StateFlow<DetailState> = _state.asStateFlow()

    fun loadDetail(context: Context) {
        viewModelScope.launch(Dispatchers.IO) {
                                            val    problem    =
ProblemRepository.getProblemById(problemId)

            if (problem == null) {
                _state.update { it.copy(isNotFound = true) }
                return@launch
            }

                                          val    codeText    =
context.resources.openRawResource(problem.solutionCode).buffer
edReader().use { it.readText() }

              Timber.d("DetailViewModel initialized for module
$moduleName : $problem")

            _state.update {
                it.copy(
```

```
                    titleRes = problem.title,
                    descRes = problem.description,
                    imgRes = problem.img,
                    code = codeText,
                )
            }
        }
    }
}
```

Tabel 5. DetailViewModelFactory Modul 4

```
package com.mobil.modul4compose.ui.screens.detail

import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelProvider

class DetailViewModelFactory(
    private val problemId: String,
    private val moduleName: String
) : ViewModelProvider.Factory {
    override fun <T : ViewModel> create(modelClass: Class<T>):
T {
                                                        if
(modelClass.isAssignableFrom(DetailViewModel::class.java)) {
            return DetailViewModel(problemId, moduleName) as T
        }
            throw  IllegalArgumentException("Unknown  ViewModel
class")
```

```
    }
}
```

Tabel 6. HomeState Modul 4

```
package
com.mobil.modul4{xml/compose}.ui.{fragments/screens}.home


import com.mobil.modul4xml.data.CodeforcesProblem


data class HomeState(
    val problems: List<CodeforcesProblem> = emptyList(),
    val navigateToDetailEvent: String? = null,
    val openExternalUrlEvent: String? = null
)
```

Tabel 7. HomeViewModel Modul 4

```
package com.mobil.modul4compose.ui.screens.home


import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.mobil.modul4compose.data.ProblemRepository
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.update
import kotlinx.coroutines.launch
```

```
import timber.log.Timber


class HomeViewModel(
    private val repository: ProblemRepository,
    private val moduleName: String
) : ViewModel() {


    private val _uiState = MutableStateFlow(HomeState())
    val uiState: StateFlow<HomeState> = _uiState.asStateFlow()


    init {
                Timber.d("ViewModel  initialized  for  module:
$moduleName")
        loadProblems()
    }


    private fun loadProblems() {
        viewModelScope.launch(Dispatchers.IO) {
            val list = repository.getAllProblems()
               Timber.d("Loaded ${list.size} problems into the
list")


            list.forEach { problem ->
                Timber.d("Problem Data : $problem")
            }


            _uiState.update { it.copy(problems = list) }
        }
```

```
    }


    fun onProblemClicked(problemId: String) {
            _uiState.update { it.copy(navigateToDetailEvent =
problemId) }
    }


    fun onExternalUrlClicked(url: String) {
             _uiState.update { it.copy(openExternalUrlEvent =
url) }
    }


    fun onDetailNavigationHandled() {
            _uiState.update { it.copy(navigateToDetailEvent =
null) }
    }


    fun onExternalUrlHandled() {
        _uiState.update { it.copy(openExternalUrlEvent = null)
}
    }
}
```

Tabel 8. HomeViewModelFactory Modul 4

```
package com.mobil.modul4compose.ui.screens.home


import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelProvider
```

```
import com.mobil.modul4compose.data.ProblemRepository


class HomeViewModelFactory(
    private val repository: ProblemRepository,
    private val moduleName: String
) : ViewModelProvider.Factory {
    @Suppress("UNCHECKED_CAST")
    override fun <T : ViewModel> create(modelClass: Class<T>):
T {
                                                        if
(modelClass.isAssignableFrom(HomeViewModel::class.java)) {
            return HomeViewModel(repository, moduleName) as T
        }
            throw  IllegalArgumentException("Unknown  ViewModel
class")
    }
}
```

Tabel 9. LanguageModel Modul 4

```
package com.mobil.modul4xml.ui.{fragments/screens}.language


data class LanguageModel(
    val name: String,
    val tag: String,
    val isSelected: Boolean
)
```

Tabel 10. LanguageState Modul 4

```
package
com.mobil.modul4{xml/compose}.ui.{fragments/screens}.language


data class LanguageState(
    val selectedTag: String = "en",
    val languages: List<LanguageModel> = listOf(
        LanguageModel("English", "en", isSelected = true),
        LanguageModel("Indonesian", "id", isSelected = false)
    )
)
```

Tabel 11. LanguageViewModel Modul 4

```
package com.mobil.modul4xml.ui.fragments.language


import androidx.appcompat.app.AppCompatDelegate
import androidx.core.os.LocaleListCompat
import androidx.lifecycle.ViewModel
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.update


class LanguageViewModel : ViewModel() {
    private val _uiState = MutableStateFlow(LanguageState())
                val    uiState:    StateFlow<LanguageState>    =
_uiState.asStateFlow()
```

```
    init {
        loadCurrentLocale()
    }

    private fun loadCurrentLocale() {
                                        val     locales     =
AppCompatDelegate.getApplicationLocales()
                        val   tag   =   if   (!locales.isEmpty)
locales[0]?.language ?: "en" else "en"
        _uiState.update { it.copy(selectedTag = tag) }
    }

    fun onLanguageSelected(tag: String) {
        val appLocale = LocaleListCompat.forLanguageTags(tag)
        AppCompatDelegate.setApplicationLocales(appLocale)
        _uiState.update { it.copy(selectedTag = tag) }
    }
}
```

Compose

Tabel 12. DetailScreen Modul 4

```
package com.mobil.modul4compose.ui.screens.detail

import androidx.compose.foundation.Image
import androidx.compose.foundation.background
import androidx.compose.foundation.horizontalScroll
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
```

```
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.RoundedCornerShape
import
androidx.compose.foundation.text.selection.SelectionContainer
import androidx.compose.foundation.verticalScroll
import androidx.compose.material.icons.Icons
import
androidx.compose.material.icons.automirrored.filled.ArrowBack
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.material3.TopAppBar
import androidx.compose.runtime.Composable
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.platform.LocalContext
```

```
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.text.font.FontFamily
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp


@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun DetailScreen(
    viewModel: DetailViewModel,
    onBack: () -> Unit
) {
    val context = LocalContext.current
    val state by viewModel.state.collectAsState()


    viewModel.loadDetail(context)


    Scaffold(
        topBar = {
            TopAppBar(
                title = {
                     val titleText = if (state.titleRes != 0)
stringResource(state.titleRes) else ""
                    Text(titleText)
                },
                navigationIcon = {
                    IconButton(onClick = onBack) {
```

```
                        Icon(Icons.AutoMirrored.Filled.ArrowBa
ck, contentDescription = "Back")
                    }
                }
            )
        }
    ) { paddingValues ->
        Box(modifier = Modifier.padding(paddingValues)) {
            if (state.isNotFound) {
                Box(Modifier.fillMaxSize(), contentAlignment =
Alignment.Center) {
                    Text("Problem not found")
                }
            } else if (state.titleRes != 0) {
                Column(
                    modifier = Modifier
                        .fillMaxSize()
                        .padding(16.dp)
                        .verticalScroll(rememberScrollState())
                ) {
                    Image(
                                                        painter  =
painterResource(state.imgRes),
                        contentDescription = null,
                                                      modifier  =
Modifier.fillMaxWidth().height(128.dp).clip(RoundedCornerShape
(8.dp)),
                        contentScale = ContentScale.Crop
                    )
```

```
                    Text(
                        text = stringResource(state.titleRes),
                        fontWeight = FontWeight.Bold,
                        fontSize = 16.sp,
                            modifier = Modifier.padding(top =
16.dp, bottom = 16.dp)
                    )
                    Text(
                        text = stringResource(state.descRes),
                          modifier = Modifier.padding(bottom =
16.dp)
                    )
                    CodePreview(state.code)
                }
            }
        }
    }
}


@Composable
fun CodePreview(code: String) {
    val horizontalScrollState = rememberScrollState()

    SelectionContainer {
        Box(
            modifier = Modifier
                .fillMaxWidth()
                .clip(RoundedCornerShape(8.dp))
```

```
                .background(Color(0xFF1E1E1E))
                .padding(12.dp)
        ) {
            Text(
                text = code,
                color = Color(0xFFD4D4D4),
                fontFamily = FontFamily.Monospace,
                fontSize = 12.sp,
                softWrap = false,
                                                    modifier   =
Modifier.horizontalScroll(horizontalScrollState)
            )
        }
    }
}
```

Tabel 13. HomeScreen Modul 4

```
package com.mobil.modul4compose.ui.screens.home


import android.content.Intent
import androidx.compose.foundation.Image
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.PaddingValues
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
```

```
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.layout.wrapContentHeight
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Language
import androidx.compose.material3.Button
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.material3.TopAppBar
import
androidx.compose.material3.carousel.HorizontalMultiBrowseCarou
sel
import
androidx.compose.material3.carousel.rememberCarouselState
import androidx.compose.runtime.Composable
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.ui.Alignment
```

```
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.navigation.NavController
import androidx.core.net.toUri
import androidx.lifecycle.viewmodel.compose.viewModel
import com.mobil.modul4compose.R
import com.mobil.modul4compose.data.CodeforcesProblem
import com.mobil.modul4compose.navigation.RoutingNames

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun HomeScreen(
    navController: NavController,
    viewModel: HomeViewModel
) {
    val state by viewModel.uiState.collectAsState()
    val context = LocalContext.current

    LaunchedEffect(state.navigateToDetailEvent) {
        state.navigateToDetailEvent?.let { problemId ->
```

```
            Timber.d("Navigating to detail page for problemId:
$problemId")
            navController.navigate(RoutingNames.DetailScreen(p
roblemId))
            viewModel.onDetailNavigationHandled()
        }
    }

    LaunchedEffect(state.openExternalUrlEvent) {
        state.openExternalUrlEvent?.let { url ->
            Timber.d("Opening external URL: $url")
              context.startActivity(Intent(Intent.ACTION_VIEW,
url.toUri()))
            viewModel.onExternalUrlHandled()
        }
    }

    Scaffold(
        topBar = {
            TopAppBar(
                            title = { Text(stringResource(id =
R.string.app_name)) },
                actions = {
                                        IconButton(onClick =
{ navController.navigate(RoutingNames.LanguageScreen) }) {
                        Icon(
                                                imageVector =
Icons.Default.Language,
                                    contentDescription = "Change
```

```
Language"
                        )
                    }
                }
            )
        }
    ) { paddingValues ->
        HomeContent(
            problems = state.problems,
            onExternalClick = viewModel::onExternalUrlClicked,
            onDetailClick = viewModel::onProblemClicked,
            modifier = Modifier.padding(paddingValues)
        )
    }
}

@Composable
private fun HomeContent(
    problems: List<CodeforcesProblem>,
    onExternalClick: (String) -> Unit,
    onDetailClick: (String) -> Unit,
    modifier: Modifier = Modifier
) {
    LazyColumn(
        modifier = modifier.fillMaxSize(),
        contentPadding = PaddingValues(bottom = 16.dp)
    ) {
        if (problems.isNotEmpty()) {
```

```
            item {
                ProblemCarousel(
                    problems,
                    onDetailClick
                )
            }

            items(problems) { problem ->
                ProblemCard(
                    problem = problem,
                                                onExternalClick  =
{ onExternalClick(problem.url) },
                                                  onDetailClick  =
{ onDetailClick(problem.problemId) }
                )
            }
        }
    }
}


@Composable
fun ProblemCard(
    problem: CodeforcesProblem,
    onExternalClick: () -> Unit,
    onDetailClick: () -> Unit
) {
    Card(
            colors  =  CardDefaults.cardColors(containerColor  =
```

```
MaterialTheme.colorScheme.surfaceVariant),
        modifier = Modifier.padding(8.dp)
    ) {
        Row(
            verticalAlignment = Alignment.CenterVertically,
            modifier = Modifier.padding(8.dp)
        ) {
            Image(
                painter = painterResource(id = problem.img),
                contentDescription = null,
                contentScale = ContentScale.Crop,
                                                    modifier   =
Modifier.width(128.dp).height(256.dp).clip(RoundedCornerShape(
16.dp))
            )
            Column {
                Text(
                    text = stringResource(problem.title),
                    modifier = Modifier.padding(16.dp),
                    fontWeight = FontWeight.Bold
                )
                Text(
                                                        text   =
stringResource(problem.description),
                    modifier = Modifier.padding(16.dp),
                    textAlign = TextAlign.Justify,
                    fontSize = 16.sp
                )
```

```
                    Row(modifier = Modifier.fillMaxWidth(),
horizontalArrangement = Arrangement.End) {
                    Button(onClick = onExternalClick, modifier
= Modifier.padding(8.dp)) {
                        Text("Problem")
                    }
                    Button(onClick = onDetailClick, modifier =
Modifier.padding(8.dp)) {
                        Text("Detail")
                    }
                }
            }
        }
    }
}
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun ProblemCarousel(
    problem: List<CodeforcesProblem>,
    onItemClick: (String) -> Unit
) {
    HorizontalMultiBrowseCarousel(
        state = rememberCarouselState { problem.count() },
        modifier = Modifier
            .fillMaxWidth()
            .wrapContentHeight()
            .padding(top = 16.dp, bottom = 16.dp),
        preferredItemWidth = 372.dp,
```

```
        itemSpacing = 8.dp,
        contentPadding = PaddingValues(horizontal = 16.dp)
    ) { i ->
        val item = problem[i]
        Image(
            modifier = Modifier
                .height(205.dp)
                .clickable { onItemClick(item.problemId) }
                .maskClip(MaterialTheme.shapes.extraLarge)
                .fillMaxSize(),
            painter = painterResource(item.img),
            contentDescription = stringResource(item.title),
            contentScale = ContentScale.Crop
        )
    }
}
```

Tabel 14. LanguageScreen Modul 4

```
package com.mobil.modul4compose.ui.screens.language


import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.material.icons.Icons
```

```
import
androidx.compose.material.icons.automirrored.filled.ArrowBack
import androidx.compose.material.icons.filled.Check
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.material3.TopAppBar
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.unit.dp
import androidx.lifecycle.viewmodel.compose.viewModel
import androidx.navigation.NavController
import com.mobil.modul4compose.R


@Composable
fun LanguageScreen(
    navController: NavController,
    viewModel: LanguageViewModel = viewModel()
) {
    val state by viewModel.uiState.collectAsState()


    LanguageContent(
```

```
        state = state,
        onBackClick = { navController.popBackStack() },
        onLanguageClick = { viewModel.onLanguageSelected(it) }
    )
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
private fun LanguageContent(
    state: LanguageState,
    onBackClick: () -> Unit,
    onLanguageClick: (String) -> Unit
) {
    Scaffold(
        topBar = {
            TopAppBar(
                            title = { Text(stringResource(id =
R.string.language_title)) },
                navigationIcon = {
                    IconButton(onClick = onBackClick) {
                        Icon(Icons.AutoMirrored.Filled.ArrowBa
ck, contentDescription = "Back")
                    }
                }
            )
        }
    ) { paddingValues ->
        Column(
```

```
            modifier = Modifier
                .fillMaxSize()
                .padding(paddingValues)
        ) {
            state.languages.forEach { language ->
                LanguageItem(
                    title = language.name,
                    tag = language.tag,
                    currentTag = state.selectedTag,
                                                    onClick   =
{ onLanguageClick(language.tag) }
                )
            }
        }
    }
}


@Composable
private fun LanguageItem(
    title: String,
    tag: String,
    currentTag: String,
    onClick: () -> Unit
) {
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .clickable(onClick = onClick)
```

```
            .padding(16.dp),
        horizontalArrangement = Arrangement.SpaceBetween,
        verticalAlignment = Alignment.CenterVertically
    ) {
        Text(text = title)
        if (currentTag == tag) {
                Icon(Icons.Default.Check, contentDescription =
"Selected")
        }
    }
}
```

Tabel 15. AppNavigation Modul 4

```
package com.mobil.modul4compose.navigation


import androidx.compose.runtime.Composable
import androidx.compose.runtime.remember
import androidx.lifecycle.viewmodel.compose.viewModel
import androidx.navigation.NavHostController
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.toRoute
import com.mobil.modul4compose.data.ProblemRepository
import com.mobil.modul4compose.ui.screens.detail.DetailScreen
import
com.mobil.modul4compose.ui.screens.detail.DetailViewModel
import
com.mobil.modul4compose.ui.screens.detail.DetailViewModelFacto
```

```
ry
import com.mobil.modul4compose.ui.screens.home.HomeScreen
import com.mobil.modul4compose.ui.screens.home.HomeViewModel
import
com.mobil.modul4compose.ui.screens.home.HomeViewModelFactory
import
com.mobil.modul4compose.ui.screens.language.LanguageScreen
import
com.mobil.modul4compose.ui.screens.language.LanguageViewModel


@Composable
fun AppNavigation(navController: NavHostController) {
    NavHost(navController = navController, startDestination =
RoutingNames.HomeScreen) {
        composable<RoutingNames.HomeScreen>{
                                    val   factory   =   remember
{ HomeViewModelFactory(ProblemRepository, "Modul4Compose") }
            val viewModel: HomeViewModel = viewModel(factory =
factory)
            HomeScreen(navController, viewModel)
        }
        composable<RoutingNames.DetailScreen>{ navBackStack ->
                                           val   detailArgs   =
navBackStack.toRoute<RoutingNames.DetailScreen>()
                                    val   factory   =   remember
{                 DetailViewModelFactory(detailArgs.problemId,
"Modul4Compose") }
            val viewModel: DetailViewModel = viewModel(factory
= factory)
```

```
            DetailScreen(viewModel,
                { navController.popBackStack() }
            )
        }
        composable<RoutingNames.LanguageScreen> {
            val viewModel: LanguageViewModel = viewModel()
            LanguageScreen(navController, viewModel)
        }
    }
}
```

Tabel 16. RoutingNames Modul 4

```
package com.mobil.modul4compose.navigation

import kotlinx.serialization.Serializable

@Serializable
sealed class RoutingNames {
    @Serializable
    object HomeScreen : RoutingNames()

    @Serializable
    data class DetailScreen(
        val problemId: String
    ) : RoutingNames()

    @Serializable
```

```
    object LanguageScreen: RoutingNames()
}
```

Tabel 17. MainActivity Compose Modul 4

```
package com.mobil.modul4compose


import android.os.Bundle
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import androidx.navigation.compose.rememberNavController
import com.mobil.modul4compose.navigation.AppNavigation
import com.mobil.modul4compose.ui.theme.modul4ComposeTheme


class MainActivity : AppCompatActivity() {


    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        plant(Timber.DebugTree())
        setContent {
            modul4ComposeTheme {
                val navController = rememberNavController()
                AppNavigation(navController)
            }
        }
    }
```

```
}
```

XML

Tabel 18. DetailFragment Modul 4

```
package com.mobil.modul4xml.ui.fragments.detail


import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.fragment.app.Fragment
import androidx.fragment.app.viewModels
import androidx.lifecycle.Lifecycle
import androidx.lifecycle.lifecycleScope
import androidx.lifecycle.repeatOnLifecycle
import androidx.navigation.fragment.findNavController
import com.mobil.modul4xml.data.ProblemRepository
import com.mobil.modul4xml.databinding.FragmentDetailBinding
import kotlinx.coroutines.launch


class DetailFragment : Fragment() {


    private var _binding: FragmentDetailBinding? = null
    private val binding get() = _binding!!
    private val viewModel: DetailViewModel by viewModels {
        val problemId = arguments?.getString("problemId")
                ?: throw IllegalArgumentException("problemId
```

```
argument is required")

        DetailViewModelFactory(problemId, "Modul 4 XML")
    }

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
            _binding = FragmentDetailBinding.inflate(inflater,
container, false)
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState:
Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        setupToolbar()
        observeViewModel()

        viewModel.loadDetail(requireContext())
    }

    private fun setupToolbar() {
        binding.topAppBar.setNavigationOnClickListener {
            findNavController().navigateUp()
```

```
        }
    }

    private fun observeViewModel() {
        viewLifecycleOwner.lifecycleScope.launch {
            viewLifecycleOwner.repeatOnLifecycle(Lifecycle.Sta
te.STARTED) {
                viewModel.state.collect { state ->
                    if (state.isNotFound) {
                                binding.tvNotFound.visibility =
View.VISIBLE
                        binding.contentScrollView.visibility =
View.GONE
                        binding.topAppBar.title = ""
                    } else if (state.titleRes != 0) {
                                binding.tvNotFound.visibility =
View.GONE
                        binding.contentScrollView.visibility =
View.VISIBLE

                                        binding.topAppBar.title =
getString(state.titleRes)
                        binding.tvTitle.setText(state.titleRes
)
                        binding.tvDescription.setText(state.de
scRes)
                        binding.tvCode.text = state.code

                        if (state.imgRes != 0) {
```

```
                                binding.ivHeader.setImageResource(
state.imgRes)
                            }
                        }
                    }
                }
            }

    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null
    }
}
```

Tabel 19. ProblemListAdapter Modul 4

```
package com.mobil.modul4xml.ui.fragments.home


import android.view.LayoutInflater
import android.view.ViewGroup
import androidx.recyclerview.widget.ListAdapter
import androidx.recyclerview.widget.RecyclerView
import com.mobil.modul4xml.databinding.ItemProblemCardBinding
import com.mobil.modul4xml.data.CodeforcesProblem
import com.mobil.modul4xml.util.DiffCallback


class ProblemListAdapter(
    private val onExternalClick: (String) -> Unit,
```

```
    private val onDetailClick: (String) -> Unit
) : ListAdapter<CodeforcesProblem, ProblemListAdapter.ProblemViewHolder>(
    DiffCallback<CodeforcesProblem> {it.problemId}
) {

    class ProblemViewHolder(
        private val binding: ItemProblemCardBinding,
        private val onExternalClick: (String) -> Unit,
        private val onDetailClick: (String) -> Unit
    ) : RecyclerView.ViewHolder(binding.root) {

        fun bind(problem: CodeforcesProblem) {
            binding.ivProblem.setImageResource(problem.img)
            binding.tvTitle.setText(problem.title)
            binding.tvDescription.setText(problem.description)

                        binding.btnProblem.setOnClickListener { onExternalClick(problem.url) }
                         binding.btnDetail.setOnClickListener { onDetailClick(problem.problemId) }
        }
    }

        override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): ProblemViewHolder {
        val binding = ItemProblemCardBinding.inflate(
            LayoutInflater.from(parent.context), parent, false
```

```
        )
            return ProblemViewHolder(binding, onExternalClick,
onDetailClick)
    }

     override fun onBindViewHolder(holder: ProblemViewHolder,
position: Int) {
        holder.bind(getItem(position))
    }
}
```

Tabel 20. CarouselAdapter Modul 4

```
package com.mobil.modul4xml.ui.fragments.home

import android.view.LayoutInflater
import android.view.ViewGroup
import androidx.recyclerview.widget.ListAdapter
import androidx.recyclerview.widget.RecyclerView
import com.mobil.modul4xml.data.CodeforcesProblem
import
com.mobil.modul4xml.databinding.ItemProblemCarouselBinding
import com.mobil.modul4xml.util.DiffCallback

class CarouselAdapter(
    private val onItemClick: (String) -> Unit
)               :               ListAdapter<CodeforcesProblem,
CarouselAdapter.CarouselViewHolder>(
    DiffCallback<CodeforcesProblem> {it.problemId}
```

```
) {

    class CarouselViewHolder(
        private val binding: ItemProblemCarouselBinding,
        private val onItemClick: (String) -> Unit
    ) : RecyclerView.ViewHolder(binding.root) {

        fun bind(problem: CodeforcesProblem) {
            binding.ivCarouselItem.setImageResource(problem.im
g)
                            binding.root.contentDescription  =
binding.root.context.getString(problem.title)

                                binding.root.setOnClickListener
{ onItemClick(problem.problemId) }
        }
    }

        override fun onCreateViewHolder(parent: ViewGroup,
viewType: Int): CarouselViewHolder {
        val binding = ItemProblemCarouselBinding.inflate(
            LayoutInflater.from(parent.context), parent, false
        )
        return CarouselViewHolder(binding, onItemClick)
    }

    override fun onBindViewHolder(holder: CarouselViewHolder,
position: Int) {
        holder.bind(getItem(position))
```

```
    }
}
```

Tabel 21. HomeFragment Modul 4

```
package com.mobil.modul4xml.ui.fragments.home


import android.content.Intent
import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.fragment.app.Fragment
import androidx.fragment.app.viewModels
import androidx.lifecycle.Lifecycle
import androidx.lifecycle.lifecycleScope
import androidx.lifecycle.repeatOnLifecycle
import androidx.navigation.fragment.findNavController
import com.mobil.modul4xml.R
import com.mobil.modul4xml.databinding.FragmentHomeBinding
import kotlinx.coroutines.launch
import androidx.core.net.toUri
import
com.google.android.material.carousel.CarouselLayoutManager
import com.google.android.material.carousel.CarouselSnapHelper
import
com.google.android.material.carousel.HeroCarouselStrategy
import com.mobil.modul4xml.data.ProblemRepository
import timber.log.Timber
```

```
class HomeFragment : Fragment() {
    private var _binding: FragmentHomeBinding? = null
    private val binding get() = _binding!!

    private val repository = ProblemRepository

      private val factory = HomeViewModelFactory(repository,
"Modul 4 XML")
       private val viewModel: HomeViewModel by viewModels
{factory}

    private lateinit var carouselAdapter: CarouselAdapter
    private lateinit var listAdapter: ProblemListAdapter

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
            _binding = FragmentHomeBinding.inflate(inflater,
container, false)
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState:
Bundle?) {
        super.onViewCreated(view, savedInstanceState)
```

```
        setupToolbar()
        setupRecyclerViews()
        observeViewModel()
    }

    private fun setupToolbar() {
                  binding.topAppBar.setOnMenuItemClickListener
{ menuItem ->
            when (menuItem.itemId) {
                R.id.action_language -> {
                    findNavController().navigate(R.id.action_h
omeFragment_to_languageFragment)
TRUE
                }
                else -> false
            }
        }
    }

    private fun setupRecyclerViews() {
                              val    carouselLayoutManager    =
CarouselLayoutManager(HeroCarouselStrategy())
                    carouselLayoutManager.carouselAlignment   =
CarouselLayoutManager.ALIGNMENT_START
                 binding.carouselRecyclerView.layoutManager  =
carouselLayoutManager

        val snapHelper = CarouselSnapHelper()
```

```
        snapHelper.attachToRecyclerView(binding.carouselRecycl
erView)

        carouselAdapter = CarouselAdapter { problemId ->
            viewModel.onProblemClicked(problemId)
        }
        binding.carouselRecyclerView.adapter = carouselAdapter

        listAdapter = ProblemListAdapter(
            onExternalClick = { url ->
                viewModel.onExternalUrlClicked(url)
            },
            onDetailClick = { problemId ->
                viewModel.onProblemClicked(problemId)
            }
        )
        binding.listRecyclerView.adapter = listAdapter
    }

    private fun navigateToDetail(problemId: String) {
        val bundle = Bundle().apply{
            putString("problemId", problemId)
        }
        findNavController().navigate(R.id.action_homeFragment_
to_detailFragment, bundle)
    }

    private fun observeViewModel() {
```

```
        viewLifecycleOwner.lifecycleScope.launch {
            viewLifecycleOwner.repeatOnLifecycle(Lifecycle.Sta
te.STARTED) {
                viewModel.uiState.collect { state ->
                    val problems = state.problems
                    carouselAdapter.submitList(problems)
                    listAdapter.submitList(problems)

                    if (problems.isEmpty()) {
                        binding.carouselRecyclerView.visibilit
y = View.GONE
                         binding.listRecyclerView.visibility =
View.GONE
                    } else {
                        binding.carouselRecyclerView.visibilit
y = View.VISIBLE
                         binding.listRecyclerView.visibility =
View.VISIBLE
                    }

                                state.navigateToDetailEvent?.let
{ problemId ->
                           Timber.d("Navigating to detail page
for problemId: $problemId")
                        navigateToDetail(problemId)
                        viewModel.onDetailNavigationHandled()
                    }

                    state.openExternalUrlEvent?.let { url ->
```

```
                        Timber.d("Opening external URL: $url")
                                                    val intent =
Intent(Intent.ACTION_VIEW, url.toUri())
                        startActivity(intent)
                        viewModel.onExternalUrlHandled()
                    }
                }
            }
        }

    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null
    }
}
```

Tabel 22. LanguageAdapter Modul 4

```
package com.mobil.modul4xml.ui.fragments.language


import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.recyclerview.widget.ListAdapter
import androidx.recyclerview.widget.RecyclerView
import com.mobil.modul4xml.databinding.ItemLanguageBinding
import com.mobil.modul4xml.util.DiffCallback
```

```
class LanguageAdapter(
    private val onLanguageClick: (String) -> Unit
) : ListAdapter<LanguageModel, LanguageAdapter.LanguageViewHolder>(
    DiffCallback<LanguageModel> { it.tag }
) {
        class LanguageViewHolder(private val binding: ItemLanguageBinding) :
        RecyclerView.ViewHolder(binding.root) {

            fun bind(item: LanguageModel, onLanguageClick: (String) -> Unit) {
            binding.tvLanguageName.text = item.name
             binding.ivCheck.visibility = if (item.isSelected) View.VISIBLE else View.INVISIBLE

            binding.root.setOnClickListener {
                onLanguageClick(item.tag)
            }
        }
    }

        override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): LanguageViewHolder {
        val binding = ItemLanguageBinding.inflate(
            LayoutInflater.from(parent.context), parent, false
        )
        return LanguageViewHolder(binding)
```

```
    }

     override fun onBindViewHolder(holder: LanguageViewHolder,
position: Int) {
        holder.bind(getItem(position), onLanguageClick)
    }
}
```

Tabel 23. LanguageFragment Modul 4

```
package com.mobil.modul4xml.ui.fragments.language

import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.fragment.app.Fragment
import androidx.fragment.app.viewModels
import androidx.lifecycle.Lifecycle
import androidx.lifecycle.lifecycleScope
import androidx.lifecycle.repeatOnLifecycle
import androidx.navigation.fragment.findNavController
import com.mobil.modul4xml.databinding.FragmentLanguageBinding
import kotlinx.coroutines.launch

class LanguageFragment : Fragment() {

    private var _binding: FragmentLanguageBinding? = null
    private val binding get() = _binding!!
```

```
    private val viewModel: LanguageViewModel by viewModels()
    private lateinit var languageAdapter: LanguageAdapter

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
          _binding = FragmentLanguageBinding.inflate(inflater,
container, false)
        return binding.root
    }

    override fun onViewCreated(view: View, savedInstanceState:
Bundle?) {
        super.onViewCreated(view, savedInstanceState)

        setupToolbar()
        setupRecyclerView()
        observeViewModel()
    }

    private fun setupToolbar() {
        binding.topAppBar.setNavigationOnClickListener {
            findNavController().navigateUp()
        }
```

```
    }

    private fun setupRecyclerView() {
        languageAdapter = LanguageAdapter { tag ->
            viewModel.onLanguageSelected(tag)
        }
        binding.rvLanguages.adapter = languageAdapter
    }

    private fun observeViewModel() {
        viewLifecycleOwner.lifecycleScope.launch {
            viewLifecycleOwner.repeatOnLifecycle(Lifecycle.Sta
te.STARTED) {
                viewModel.uiState.collect { state ->
                            val uiModels = state.languages.map
{ language ->
                        LanguageModel(
                            name = language.name,
                            tag = language.tag,
                                  isSelected = language.tag ==
state.selectedTag
                        )
                    }
                    languageAdapter.submitList(uiModels)
                }
            }
        }
    }
```

```
    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null
    }
}
```

Tabel 24. DiffCallback Modul 4

```
package com.mobil.modul4xml.util

import androidx.recyclerview.widget.DiffUtil

class DiffCallback<T : Any>(
    private val itemIdentifier: (T) -> Any
) : DiffUtil.ItemCallback<T>() {

      override fun areItemsTheSame(oldItem: T, newItem: T):
Boolean {
                          return   itemIdentifier(oldItem)   ==
itemIdentifier(newItem)
    }

     override fun areContentsTheSame(oldItem: T, newItem: T):
Boolean {
        return oldItem == newItem
    }
}
```

Tabel 25. MainActivity XML Modul 4

```
package com.mobil.modul4xml


import android.os.Bundle
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat
import com.mobil.modul4xml.databinding.ActivityMainBinding


class MainActivity : AppCompatActivity() {


    private lateinit var binding: ActivityMainBinding


    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()


        plant(debugTree())


        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)


        ViewCompat.setOnApplyWindowInsetsListener(binding.root
) { v, insets ->
                                        val    systemBars    =
insets.getInsets(WindowInsetsCompat.Type.systemBars())
                v.setPadding(systemBars.left,  systemBars.top,
```

```
systemBars.right, systemBars.bottom)
            insets
        }
    }
}
```

XML Layout

Tabel 26. MainActivity XML Modul 4

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <androidx.fragment.app.FragmentContainerView
        android:id="@+id/nav_host_fragment"
        android:name="androidx.navigation.fragment.NavHostFrag
ment"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        app:defaultNavHost="true"
        app:navGraph="@navigation/nav_graph" />

</LinearLayout>
```

Tabel 27. FragmentDetail XML Modul 4

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".ui.fragments.detail.DetailFragment">

    <com.google.android.material.appbar.AppBarLayout
        android:id="@+id/appBarLayout"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent">

        <com.google.android.material.appbar.MaterialToolbar
            android:id="@+id/topAppBar"
            android:layout_width="match_parent"
            android:layout_height="?attr/actionBarSize"
            app:navigationIcon="@drawable/ic_arrow_back"
            tools:title="Problem Title" />
    </com.google.android.material.appbar.AppBarLayout>

    <TextView
        android:id="@+id/tvNotFound"
```

```
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/problem_not_found"
        android:visibility="gone"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@id/appBarLayout"
        tools:visibility="visible" />

    <androidx.core.widget.NestedScrollView
        android:id="@+id/contentScrollView"
        android:layout_width="0dp"
        android:layout_height="0dp"
        android:padding="16dp"
        android:visibility="gone"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@id/appBarLayout"
        tools:visibility="visible">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical">

            <com.google.android.material.imageview.ShapeableIm
```

```
ageView
                android:id="@+id/ivHeader"
                android:layout_width="match_parent"
                android:layout_height="128dp"
                android:scaleType="centerCrop"
                app:shapeAppearanceOverlay="@style/ShapeAppear
anceOverlay.App.CornerSize8dp"
                tools:src="@tools:sample/backgrounds/scenic" /
>

            <TextView
                android:id="@+id/tvTitle"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:layout_marginTop="16dp"
                android:layout_marginBottom="16dp"
                android:textSize="16sp"
                android:textStyle="bold"
                tools:text="Problem Title" />

            <TextView
                android:id="@+id/tvDescription"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:layout_marginBottom="16dp"
                        tools:text="Problem description goes
here..." />
```

```
            <com.google.android.material.card.MaterialCardView
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                app:cardBackgroundColor="#1E1E1E"
                app:cardCornerRadius="8dp"
                app:strokeWidth="0dp">

                <HorizontalScrollView
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:fillViewport="true"
                    android:scrollbars="none">

                    <TextView
                        android:id="@+id/tvCode"
                        android:layout_width="wrap_content"
                        android:layout_height="wrap_content"
                        android:fontFamily="monospace"
                        android:padding="12dp"
                        android:textColor="#D4D4D4"
                        android:textIsSelectable="true"
                        android:textSize="12sp"
                                    tools:text="fun main() {\n
println() \n}" />
                </HorizontalScrollView>
            </com.google.android.material.card.MaterialCardVie
w>
```

```
        </LinearLayout>
    </androidx.core.widget.NestedScrollView>

</androidx.constraintlayout.widget.ConstraintLayout>
```

Tabel 28. FragmentHome Modul 4

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.coordinatorlayout.widget.CoordinatorLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".ui.fragments.home.HomeFragment">

    <com.google.android.material.appbar.AppBarLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content">

        <com.google.android.material.appbar.MaterialToolbar
            android:id="@+id/topAppBar"
            android:layout_width="match_parent"
            android:layout_height="?attr/actionBarSize"
            app:title="@string/app_name"
            app:menu="@menu/home_menu" />
    </com.google.android.material.appbar.AppBarLayout>
```

```
    <androidx.core.widget.NestedScrollView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        app:layout_behavior="@string/appbar_scrolling_view_beh
avior">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:paddingBottom="16dp">

            <androidx.recyclerview.widget.RecyclerView
                android:id="@+id/carouselRecyclerView"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:layout_marginTop="16dp"
                android:layout_marginBottom="16dp"
                android:clipToPadding="false"
                android:paddingHorizontal="16dp"
                tools:listitem="@layout/item_problem_carousel"
/>

            <androidx.recyclerview.widget.RecyclerView
                android:id="@+id/listRecyclerView"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:nestedScrollingEnabled="false"
```

```
                app:layoutManager="androidx.recyclerview.widge
t.LinearLayoutManager"
                tools:listitem="@layout/item_problem_card" />


        </LinearLayout>
    </androidx.core.widget.NestedScrollView>


</androidx.coordinatorlayout.widget.CoordinatorLayout>
```

Tabel 29. FragmentLanguage XML Modul 4

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context=".ui.fragments.language.LanguageFragment">

    <com.google.android.material.appbar.AppBarLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content">

        <com.google.android.material.appbar.MaterialToolbar
            android:id="@+id/topAppBar"
            android:layout_width="match_parent"
            android:layout_height="?attr/actionBarSize"
```

```
            app:navigationIcon="@drawable/ic_arrow_back"
            app:title="@string/language_title" />
    </com.google.android.material.appbar.AppBarLayout>

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/rvLanguages"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        app:layoutManager="androidx.recyclerview.widget.Linear
LayoutManager"
        tools:listitem="@layout/item_language" />

</LinearLayout>
```

Tabel 30. ItemLanguage XML Modul 4

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:background="?attr/selectableItemBackground"
    android:clickable="true"
    android:focusable="true"
    android:gravity="center_vertical"
    android:orientation="horizontal"
    android:padding="16dp">
```

```
    <TextView
        android:id="@+id/tvLanguageName"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:textAppearance="?attr/textAppearanceBodyLarge"
        tools:text="English" />

    <ImageView
        android:id="@+id/ivCheck"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:src="@drawable/ic_check"
        android:visibility="invisible"
        tools:visibility="visible" />
</LinearLayout>
```

Tabel 31. ItemProblemCard XML Modul 4

```
<?xml version="1.0" encoding="utf-8"?>
<com.google.android.material.card.MaterialCardView
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginHorizontal="8dp"
    android:layout_marginVertical="8dp"
```

```
    app:cardBackgroundColor="?attr/colorSurfaceVariant"
    app:cardCornerRadius="16dp">

    <androidx.constraintlayout.widget.ConstraintLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:padding="8dp">

        <com.google.android.material.imageview.ShapeableImageV
iew
            android:id="@+id/ivProblem"
            android:layout_width="128dp"
            android:layout_height="256dp"
            android:scaleType="centerCrop"
            app:layout_constraintBottom_toBottomOf="parent"
            app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toTopOf="parent"
            app:shapeAppearanceOverlay="@style/ShapeAppearance
Overlay.App.CornerSize16Percent"
            tools:src="@tools:sample/backgrounds/scenic" />

        <TextView
            android:id="@+id/tvTitle"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_marginStart="16dp"
            android:layout_marginTop="8dp"
            android:layout_marginEnd="16dp"
```

```
        android:textStyle="bold"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toEndOf="@id/ivProblem"
        app:layout_constraintTop_toTopOf="parent"
        tools:text="Problem Title" />

    <TextView
        android:id="@+id/tvDescription"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginStart="16dp"
        android:layout_marginTop="16dp"
        android:layout_marginEnd="16dp"
        android:justificationMode="inter_word"
        android:textSize="16sp"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toEndOf="@id/ivProblem"
        app:layout_constraintTop_toBottomOf="@id/tvTitle"
        tools:text="Problem description goes here" />

    <Button
        android:id="@+id/btnProblem"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginEnd="8dp"
        android:text="@string/problem"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toStartOf="@id/btnDetail"
```

```
            app:layout_constraintTop_toBottomOf="@id/tvDescrip
tion"
            app:layout_constraintVertical_bias="1.0" />

        <Button
            android:id="@+id/btnDetail"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="@string/detail"
            app:layout_constraintBottom_toBottomOf="parent"
            app:layout_constraintEnd_toEndOf="parent"
            app:layout_constraintTop_toBottomOf="@id/tvDescrip
tion"
            app:layout_constraintVertical_bias="1.0" />

    </androidx.constraintlayout.widget.ConstraintLayout>
</com.google.android.material.card.MaterialCardView>
```

Tabel 32. ItemProblemCarousel XML Modul 4

```
<?xml version="1.0" encoding="utf-8"?>
<com.google.android.material.carousel.MaskableFrameLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="205dp"
    android:layout_marginEnd="8dp"
    app:shapeAppearanceOverlay="@style/ShapeAppearanceOverlay.
```

```
App.CornerSizeExtraLarge">


    <ImageView
        android:id="@+id/ivCarouselItem"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:scaleType="centerCrop"
        tools:src="@tools:sample/backgrounds/scenic" />


</com.google.android.material.carousel.MaskableFrameLayout>
```

Tabel 33. HomeMenu XML Modul 4

```
<?xml version="1.0" encoding="utf-8"?>
<menu
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto">
    <item
        android:id="@+id/action_language"
        android:icon="@drawable/ic_language"
        android:title="Language"
        app:showAsAction="ifRoom" />
</menu>
```

XML Nav Graph

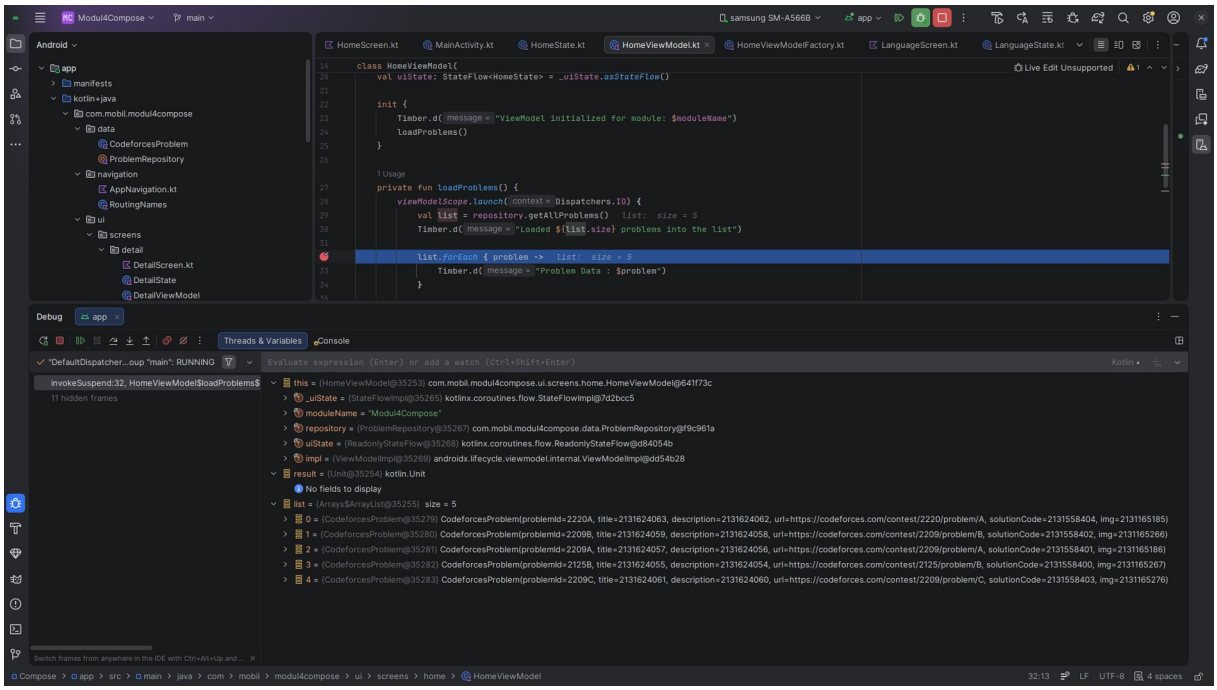
Tabel 34. NavGraph XML Modul 4

```
<?xml version="1.0" encoding="utf-8"?>
<navigation
```

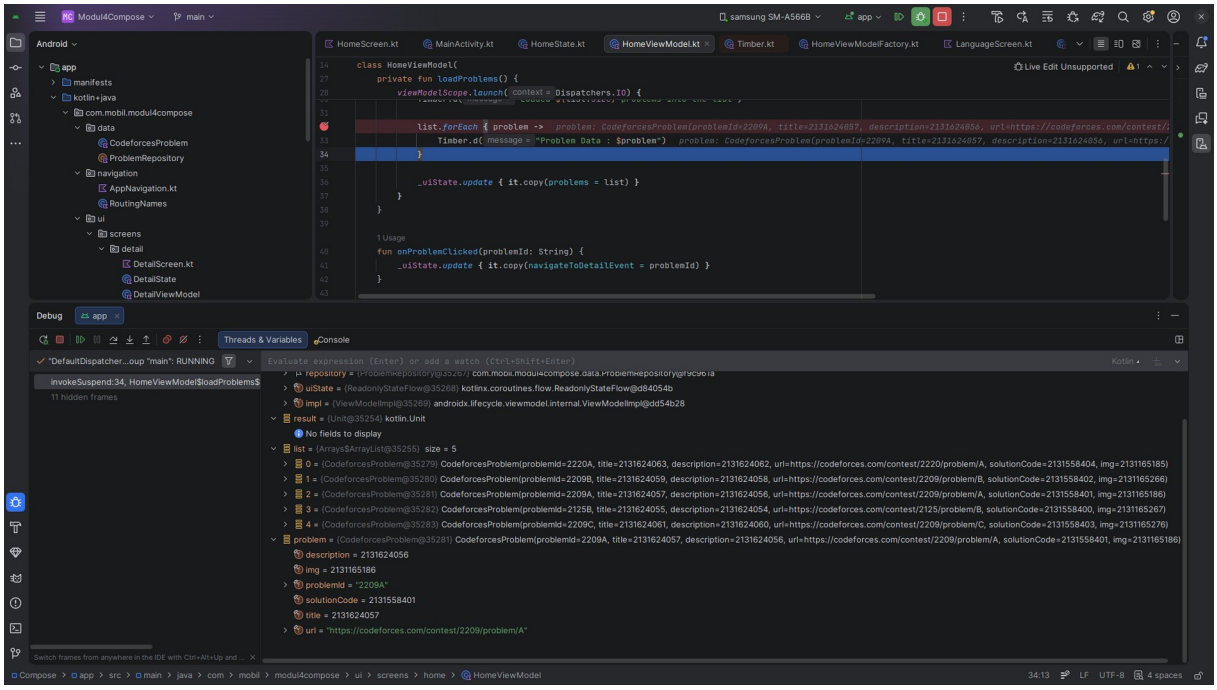
```
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/nav_graph"
    app:startDestination="@id/homeFragment">

    <fragment
        android:id="@+id/homeFragment"
        android:name="com.mobil.modul4xml.ui.fragments.home.HomeFragment"
        android:label="@string/app_name"
        tools:layout="@layout/fragment_home">

        <action
            android:id="@+id/action_homeFragment_to_detailFragment"
            app:destination="@id/detailFragment" />

        <action
            android:id="@+id/action_homeFragment_to_languageFragment"
            app:destination="@id/languageFragment" />
    </fragment>

    <fragment
        android:id="@+id/detailFragment"
        android:name="com.mobil.modul4xml.ui.fragments.detail.DetailFragment"
        android:label="Detail"
```

```
        tools:layout="@layout/fragment_detail">


        <argument
            android:name="problemId"
            app:argType="string" />
    </fragment>


    <fragment
        android:id="@+id/languageFragment"
        android:name="com.mobil.modul4xml.ui.fragments.languag
e.LanguageFragment"
        android:label="Language"
        tools:layout="@layout/fragment_language">
    </fragment>


</navigation>
```

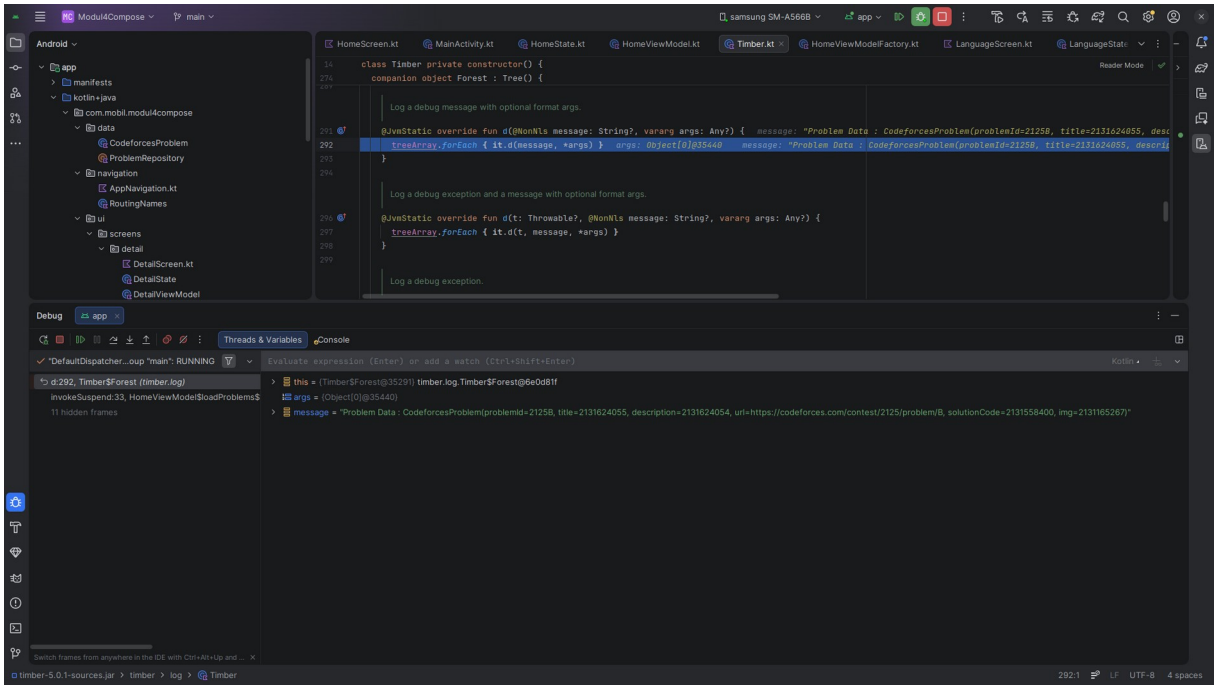
B. Output Program



Gambar 1. Screenshot Debugging HomeViewModel Modul 4



Gambar 2. Screenshot Debugging HomeViewModel step over Modul 4



Gambar 3. Screenshot Debugging HomeViewModel step into Modul 4

C. Pembahasan

Karena saya sudah mengimplementasikan MVVM dan StateFlow di modul sebelumnya jadi saya cuma menambahkan HomeViewModelFactory dan DetailViewModelFactory. Beberapa source code saya ubah seperti HomeScreen atau HomeFragment, HomeViewModel, DetailViewModel dan HomeState untuk memenuhi persyaratan laprak.

StateFlow

saya tidak membahas ini pada modul sebelumnya jadi saya akan menjelaskan secara singkat tentang StateFlow yaitu state-holder observable flow dari Kotlin Coroutines yang selalu menyimpan nilai terbaru dan akan meng-emit nilai tersebut ke semua collector ketika state berubah. StateFlow cocok digunakan MVVM karena tetap menyimpan nilai terakhir meskipun tidak ada collector yang aktif.

Factory Pattern

ViewModel tidak bisa menerima parameter ketika diinisialisasi oleh karena itulah kita memerlukan Factory Pattern sebenarnya orang-orang memakai hilt untuk hal tersebut tapi karena laprak ini meminta Factory Pattern maka akan saya penuhi. Factory Pattern adalah

sebuah design pattern untuk membuat objek dengan dependency-nya. Dalam konteks ViewModel ini, kita membuat class yang akan mengimplementasikan ViewModelProvider.Factory lalu meng-override method create untuk menginisialisasi ViewModel dengan parameter yang dibutuhkan. Sebenarnya sebagai alternative dari boilerplate Factory Pattern kita bisa menggunakan setter dan getter di ViewModel.

Click Event pada View Model

kita memisahkan logika untuk click event dari fragment/screen ke ViewModel supaya fragment/screen fokus pada UI saja atau View saja. Dengan memindahkan logika click event ke ViewModel, fragment/screen hanya perlu memanggil fungsi ViewModel ketika suatu view diklik tanpa perlu tahu apa yang terjadi di baliknya. ViewModel kemudian memproses event tersebut dan mengupdate state yang diobservasi fragment/screen untuk memperbarui UI.

Debugging

Bagaimana developer bisa memperbaiki program yangn dapat berjalan dengan lancar tapi tiba-tiba crash tanpa adanya tanda-tanda? Atau terdapat bug tertentu yang hanya akan terdeteksi ketika aplikasi dijalankan? Debugging inilah jawaban dari pertanyaann tersebut, dengan menentukan kapan program akan berhenti (jeda) untuk diperiksa memory, heap atau stack developer bisa menganalisis pada bagian mana crash tersebut akan terjadi, mencari tahu identitas bug yang menyebabkann crash dan memperbaiki crash tersebut.

Step next

Step next digunakan untuk mengeksekusi baris saat ini dan melompat ke baris berikutnya tanpa masuk ke dalam fungsi yang dipanggil.

Step in

Step in digunakan untuk masuk ke dalam fungsi yang dipanggil pada baris saat ini sehingga kita bisa menelusuri eksekusi secara lebih detail di dalam fungsi tersebut.

Step out

Step out adalah kebalikan dari step in, yaitu keluar dari fungsi yang sedang diperiksa dan kembali ke pemanggil fungsi tersebut.

SOAL 2

Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya

A. Pembahasan

Application Class adalah base class yang mewakili semua aplikasi Android dan diinisialisasi sebelum Activity, Service, atau komponen lainnya. Class ini memiliki lifecycle yang sama dengan lifecycle aplikasi android. Biasanya digunakan untuk inisialisasi global seperti library atau dependency yang dibutuhkan di seluruh aplikasi seperti dependency injection, database, atau analytics. Application class juga bisa digunakan untuk menyimpan state global yang perlu diakses dari mana saja tanpa perlu passing data antar komponen, juga mengatur konfigurasi awal untuk semua aplikasi seperti logging atau crash reporting.

Untuk menggunakan Application Class kita hanya perlu membuat class yang meng-extend Application lalu memasukkannya ke dalam AndroidManifest.xml pada atribut android:name. Application class sebaiknya tidak digunakan untuk menyimpan terlalu banyak state karena Android bisa saja mematikan proses aplikasi tiba-tiba dan state akan hilang.

LINK GITHUB

https://github.com/Gunturadhtya/Praktikum-Pemrograman-Mobile